

Backend Developer Technical Assessment

DataHarvest Inc.

Node.js · Web Scraping · Queue Management · Data Pipeline

Assessment Date: April 13, 2026

Duration: 6 – 8 hours · Difficulty: Mid-Senior Level

Version 1.0

Company Context

DataHarvest Inc. is a fictitious market-intelligence company that aggregates publicly available product and pricing data from e-commerce websites and news sources. The engineering team needs a robust, production-grade data pipeline capable of scheduling scrape jobs, processing the raw data through a queue, persisting clean records to a database, and exposing the results through a lightweight REST API.

Your mission: Design and implement the *DataHarvest Pipeline Service* — a Node.js application that orchestrates web scraping workers, manages job queues with retry and failure handling, transforms raw scraped data, and stores it for downstream consumption. The system must be observable, resilient to transient failures, and easy to extend with new data sources.

Assessment Overview

Project	DataHarvest Pipeline Service
Stack	Node.js 20 LTS · TypeScript (recommended) · PostgreSQL or SQLite
Duration	6 – 8 hours (no enforced time limit)
Submission	GitHub / GitLab repository + README + Postman collection or curl examples
Evaluation	Architecture · Scraping reliability · Queue design · Code quality · Tests

1 System Architecture

The pipeline is composed of four loosely coupled layers. Each layer must be implemented as a distinct module (or package) within the monorepo. Inter-layer communication happens exclusively through the job queue — never direct function calls across layers.

1 Scheduler	scheduler/	Triggers scrape jobs on a configurable cron schedule. Pushes job descriptors onto the P
2 Scraper Workers	workers/	Consume PENDING jobs. Fetch remote HTML/JSON. Push raw payloads onto the RAW
3 Transformer	transformer/	Consume RAW jobs. Parse, validate, and normalise data. Push clean records onto the P
4 Persister	persister/	Consume PROCESSED jobs. Upsert records into the database. Emit metrics (job count,

Queue technology: Use **BullMQ** (Redis-backed) as the queue implementation. Redis may be run locally via Docker. Alternatively, **pg-boss** (PostgreSQL-backed) is accepted if you prefer a single infrastructure dependency. Document your choice in the README.

2 Data Sources to Scrape

Implement scrapers for the following two public, scraping-friendly sources. Both are real sites with stable HTML/JSON structures. Respect `robots.txt` and add a courteous delay between requests.

Source A — Books to Scrape (books.toscrape.com)

URL: <https://books.toscrape.com> — a sandbox site explicitly designed for scraping practice.

- Scrape the first **5 catalogue pages** (50 books per run, pagination handled automatically).
- Extract per book: **title, price (GBP), star rating (1–5), availability, category, product URL**.
- Follow the product URL for each book and additionally extract: **UPC, description, number of reviews**.
- Normalise price to a float (strip the £ sign). Convert star-rating words ("Three") to integers.

Source B — Hacker News Newest (news.ycombinator.com/newest)

URL: <https://news.ycombinator.com/newest> — publicly accessible, no login required.

- Scrape the **2 most recent pages** of new submissions (~60 items).
- Extract per item: **item_id, title, URL (or self-post flag), score, author, age_text, comment_count**.
- Deduplicate by `item_id` — do not insert the same story twice.
- Skip job listings (titles starting with "Ask HN" or "Show HN" may be kept but flagged with a `type` field).

3 Queue Management Requirements

Named Queues	Define three distinct named queues: <code>scrape:pending</code> , <code>scrape:raw</code> , and <code>scrape:processed</code> . Each queue must have its own dedicated worker pool.
Job Schema	Every job must carry a typed payload. At minimum: <code>{ jobId, source, createdAt, payload, attempt }</code> . Use JSON Schema or Zod for runtime validation before processing.
Concurrency	Scraper workers: max 3 concurrent jobs. Transformer workers: max 5 concurrent jobs. Persister workers: max 2 concurrent jobs. Make these values configurable via environment variables.
Retry Policy	Failed scraper jobs: retry up to 3 times with exponential back-off (base 2 s, max 30 s). Failed transformer jobs: retry once. Failed persister jobs: retry up to 5 times with linear back-off (5 s each).
Dead Letter Queue	Jobs exceeding their retry budget must move to a <code>scrape:dlq</code> queue with the full error and stack trace preserved. Expose a GET endpoint to inspect DLQ contents.
Job Prioritisation	Source A (books) jobs use priority 2 (normal). Source B (HN) jobs use priority 1 (high). Priority 1 jobs should be processed first.
Graceful Shutdown	On SIGTERM/SIGINT the application must: stop accepting new jobs, wait for in-flight jobs to complete (max 30 s grace), then shut down cleanly. Log the shutdown sequence.
Scheduled Triggers	Source A scrape: every day at 02:00 UTC. Source B scrape: every 15 minutes. Use node-cron or a BullMQ repeatable job. Both schedules must be configurable.

4 Database Schema

Use PostgreSQL (preferred) or SQLite. Manage schema with **Knex.js** migrations or **Prisma**. The following tables are required:

--	--	--	--

scrape_jobs	id	UUID PK	
	source	VARCHAR(50)	books hackernews
	status	VARCHAR(20)	pending processing done failed
	triggered_at	TIMESTAMPTZ	
	completed_at	TIMESTAMPTZ NULL	
	error_message	TEXT NULL	
books	id	UUID PK	
	upc	VARCHAR(30) UNIQUE	Natural dedup key
	title	TEXT NOT NULL	
	price_gbp	NUMERIC(8,2)	
	rating	SMALLINT	1–5
	category	VARCHAR(80)	
	available	BOOLEAN	
	description	TEXT NULL	
	num_reviews	INTEGER	
	scraped_at	TIMESTAMPTZ	
hn_stories	id	UUID PK	
	hn_item_id	BIGINT UNIQUE	Natural dedup key
	title	TEXT NOT NULL	
	url	TEXT NULL	NULL for self-posts
	score	INTEGER	
	author	VARCHAR(100)	
	comment_count	INTEGER	
	story_type	VARCHAR(20)	story ask show job
	scraped_at	TIMESTAMPTZ	

5 REST API Endpoints

Expose a lightweight HTTP API (Express.js or Fastify) for operations and monitoring. All responses must be JSON. Use `/api/v1` as the base path.

Method	Path	Description
POST	/api/v1/jobs/trigger	Manually trigger a scrape job. Body: { source: 'books' 'hackernews' }
GET	/api/v1/jobs	List all scrape jobs. Query: ?status=&source;=&limit;=&offset;=
GET	/api/v1/jobs/:id	Get single job detail including error if failed
GET	/api/v1/jobs/dlq	Inspect dead-letter queue contents (last 50 entries)
DELETE	/api/v1/jobs/dlq/:jobId	Remove a specific job from the DLQ
POST	/api/v1/jobs/dlq/:jobId/retry	Re-queue a DLQ job for reprocessing
GET	/api/v1/books	List scraped books. Query: ?category=&minRating;=&page;=&limit;=
GET	/api/v1/books/:upc	Get a single book by UPC
GET	/api/v1/stories	List HN stories. Query: ?type=&minScore;=&page;=&limit;=
GET	/api/v1/stories/:hn_item_id	Get single story by HN item ID
GET	/api/v1/metrics	Queue metrics: depth, throughput, error rate per queue
GET	/api/v1/health	Health check: DB connection, Redis/queue connectivity

6 Non-Functional Requirements

Logging	Use Pino or Winston for structured JSON logs. Each log line must include: timestamp, level, module, jobId (when applicable), source, and message. Log every job state transition.
Configuration	All runtime parameters (DB URL, Redis URL, concurrency limits, schedules, request timeouts) must be sourced from environment variables. Provide a complete <code>.env.example</code> file.
Error Handling	Never let an unhandled promise rejection crash the process. Attach <code>process.on('unhandledRejection')</code> and <code>process.on('uncaughtException')</code> handlers that log and gracefully shut down.
Rate Limiting	Honour a configurable minimum delay between HTTP requests to the same host (default 1 000 ms). Implement a simple token-bucket or leaky-bucket in the scraper worker.
Idempotency	Re-running the same scrape job must not duplicate database records. Use UPSERT (INSERT ... ON CONFLICT DO UPDATE) keyed on the natural dedup column.

Observability

The `/api/v1/metrics` endpoint must expose: jobs completed/failed/pending per queue in the last 1 h, average processing latency, and last successful run timestamp per source.

Docker

Provide a `docker-compose.yml` that starts the app, Redis, and PostgreSQL. The app must be reachable at `http://localhost:3000` after `docker compose up`.

7 Deliverables & Repository Structure

```
dataharvest-pipeline/
├── src/
│   ├── scheduler/
│   │   ├── index.ts # Cron definitions
│   │   └── jobFactory.ts # Builds job descriptors
│   ├── workers/
│   │   ├── scraperWorker.ts # BullMQ worker: PENDING -> RAW
│   │   └── scrapers/
│   │       ├── booksScraper.ts
│   │       ├── hnScraper.ts
│   │       └── rateLimiter.ts
│   ├── transformer/
│   │   ├── transformerWorker.ts # BullMQ worker: RAW -> PROCESSED
│   │   ├── bookTransformer.ts
│   │   └── hnTransformer.ts
│   ├── persister/
│   │   ├── persisterWorker.ts # BullMQ worker: PROCESSED -> DB
│   │   └── upsert.ts
│   ├── api/
│   │   ├── server.ts
│   │   └── routes/
│   │       ├── jobs.ts
│   │       ├── books.ts
│   │       ├── stories.ts
│   │       └── metrics.ts
│   ├── middleware/
│   ├── db/
│   │   ├── client.ts
│   │   └── migrations/
│   ├── queue/
│   │   ├── queues.ts # Queue instances & shared config
│   │   └── dlq.ts
│   ├── logger.ts
│   └── index.ts # Entry point
├── tests/
│   ├── unit/
│   └── integration/
├── docker-compose.yml
├── Dockerfile
└── .env.example
```

```
■■■ package.json
■■■ tsconfig.json
■■■ README.md
```

README must include:

- Architecture diagram (ASCII art or embedded image)
- Step-by-step local setup instructions (with and without Docker)
- All environment variables and their purpose
- How to manually trigger a scrape job via curl or Postman
- How to monitor queue status
- Known limitations and trade-offs made

8 Evaluation Rubric

System Architecture	Clean layer separation, queue-mediated communication, no tight coupling	High	20
Scraper Quality	Correct data extraction, pagination, rate limiting, resilience to missing fields	High	18
Queue Design	Retry policies, DLQ, priorities, concurrency, graceful shutdown	High	18
Data Persistence	Correct schema, migrations, upsert idempotency, query correctness	Medium	12
REST API	RESTful conventions, filtering/pagination, error responses, health endpoint	Medium	10
Observability & Logging	Structured logs, job state transitions, metrics endpoint populated	Medium	8
Code Quality	TypeScript types, naming, no dead code, env-var config, ESLint clean	Medium	7
Tests	Unit tests for transformers/validators; at least 1 integration test	Medium	5
Documentation & DevEx	README completeness, docker-compose works, .env.example provided	Low	2
Total			100

9 Bonus Challenges (Optional)

The following items are **not required** but will distinguish your submission. Each carries up to 5 bonus points.

★ Playwright / Puppeteer Scraper	Implement a headless-browser fallback for Source A that activates when the cheerio-based scraper detects a CAPTCHA or JS-rendered page.
★ BullMQ Dashboard	Integrate Bull Board or equivalent and expose it at <code>/admin/queues</code> (protected by basic auth).
★ Full Test Coverage	Achieve >70% line coverage with Jest or Vitest, including integration tests that spin up a real Redis and test-DB instance via Testcontainers.
★ Change Data Capture	Emit a webhook (or SSE stream) whenever a new book or story is inserted, enabling downstream consumers to react in real time.
★ OpenAPI Spec	Generate a fully documented OpenAPI 3.1 spec from your route definitions. Serve it at <code>/api/docs</code> via Swagger UI.
★ Metrics Endpoint (Prometheus format)	Expose <code>/metrics</code> in Prometheus text format so the pipeline can be scraped by a Prometheus server.

10 Rules, Tips & Process

- **Original work:** You may use official docs, npm package READMEs, and Stack Overflow for reference, but the implementation must be your own. Do not fork existing pipeline projects.
- **Commit hygiene:** Commit incrementally with descriptive messages. A single "init" commit covering everything will be treated as a red flag.
- **Scraping etiquette:** Add a User-Agent header identifying the request as a technical-assessment bot (e.g. `DataHarvestBot/1.0 (tech-assessment)`). Respect any per-host rate limiting in your implementation.
- **Secrets:** No API keys, tokens, or database passwords may appear in the repository. Use environment variables and add `.env` to `.gitignore`.
- **TypeScript:** Strongly recommended. JavaScript is accepted but TypeScript demonstrates production-grade habits. If using TS, set `strict: true` in `tsconfig.json`.
- **Questions:** Clarifying questions are encouraged and should be sent to your recruiting contact. The ability to ask good questions is part of the assessment.
- **After submission:** Shortlisted candidates will be invited to a 45-minute technical review where they will be asked to walk through their design, justify trade-offs, and add a small live feature.